

Módulos de Software Codificados y Probados**GA7-220501096-AA2-EV02****Autor:**

- Yeison Adolfo Díaz Tapasco

Institución:

Servicio Nacional de Aprendizaje (SENA)

Programa: Análisis y Desarrollo de Software

Profesor:

Oscar Fernando Carvajal

Mayo 2026

TABLA DE CONTENIDO

1. Introducción
2. Objetivos
3. Herramientas Utilizadas
4. Descripción del Proyecto
5. Estructura del Proyecto
6. Base de Datos
7. Desarrollo del Sistema
 - 7.1 registrar.jsp
 - 7.2 VehiculoServlet.java
 - 7.3 VehiculoDAO.java
 - 7.4 ConexionBD.java
 - 7.5 resultado.jsp
8. Ejecución del Sistema
9. Dificultades Presentadas
10. Solución Implementada
11. Resultados Obtenidos
12. Conclusiones

1. INTRODUCCIÓN

En la presente evidencia se desarrolla una aplicación web orientada al registro de vehículos para un sistema de parqueadero utilizando tecnologías Java Web. El proyecto fue construido mediante JSP, Servlets, Maven, Apache Tomcat y MySQL, aplicando conceptos de programación orientada a objetos, arquitectura MVC y conexión a bases de datos.

El sistema permite registrar información de vehículos mediante un formulario web, procesar los datos utilizando Servlets y almacenarlos en una base de datos MySQL.

Posteriormente, los datos registrados son visualizados en pantalla confirmando el correcto funcionamiento del sistema.

Durante el desarrollo se configuró el entorno de trabajo utilizando Apache NetBeans, Tomcat 11, Maven y phpMyAdmin para la administración de la base de datos.

2. OBJETIVOS

2.1 Objetivo General

Desarrollar una aplicación web funcional para el registro de vehículos utilizando tecnologías Java Web y conexión a bases de datos MySQL.

2.2 Objetivos Específicos

- Implementar formularios JSP para captura de información.
- Utilizar Servlets para el procesamiento de solicitudes HTTP.
- Conectar la aplicación web con una base de datos MySQL.
- Aplicar una arquitectura MVC básica.
- Desplegar el proyecto correctamente en Apache Tomcat.
- Validar el almacenamiento de datos en la base de datos.

3. HERRAMIENTAS UTILIZADAS

Herramienta	Función
Apache NetBeans	Entorno de desarrollo
Java JDK 17	Lenguaje de programación
Maven	Gestión del proyecto
Apache Tomcat 11	Servidor web
MySQL	Base de datos
phpMyAdmin	Administración de BD
XAMPP	Servicio MySQL
JSP	Interfaz web
Servlets	Procesamiento de datos

4. DESCRIPCIÓN DEL PROYECTO

El sistema desarrollado corresponde a una aplicación web para la gestión básica de ingreso de vehículos a un parqueadero. El aplicativo permite registrar la placa del vehículo, el tipo y la hora de entrada mediante un formulario web.

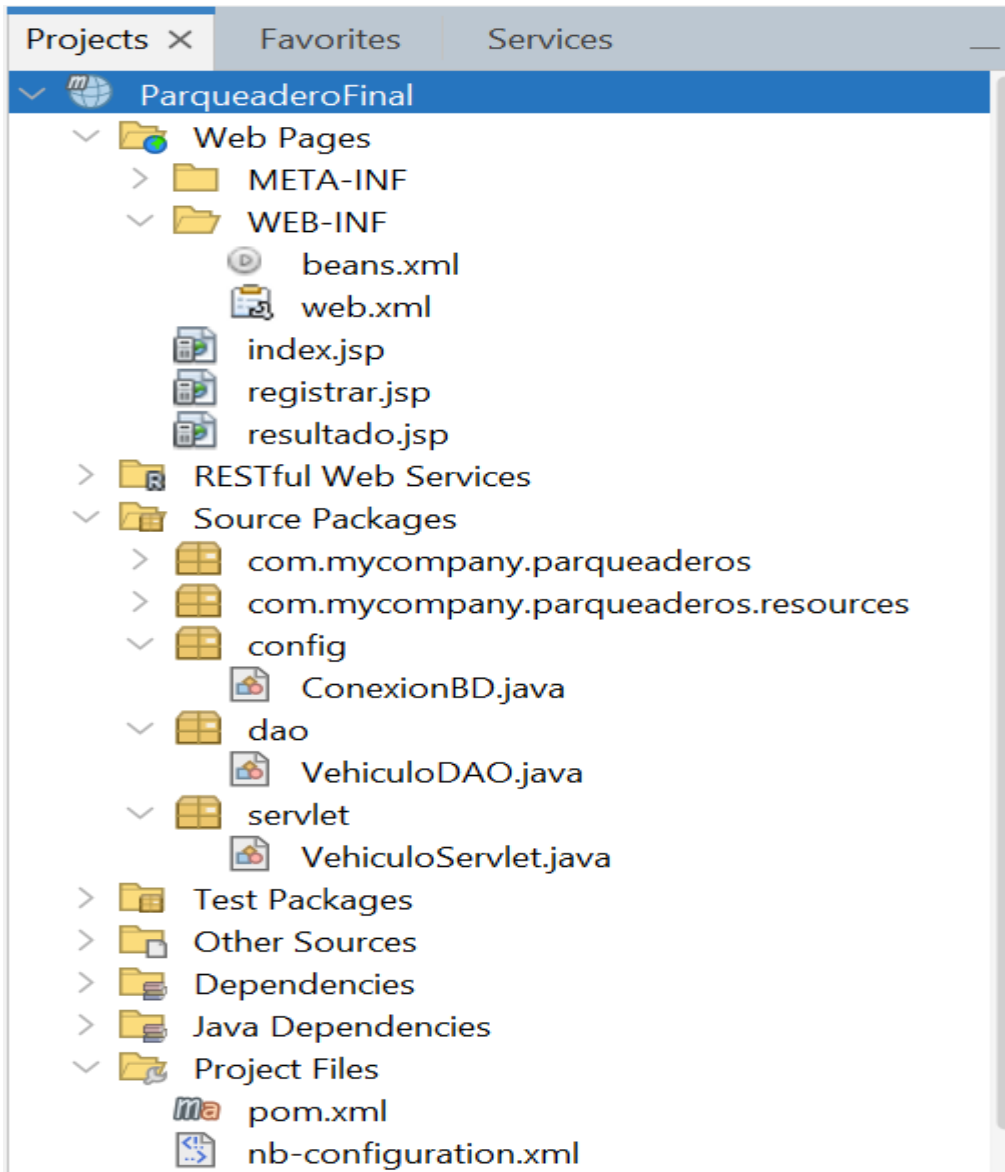
La información es enviada al Servlet mediante el método POST. Posteriormente el Servlet procesa los datos y los envía a la capa DAO encargada de almacenar la información en MySQL.

Finalmente el sistema muestra una página de confirmación con los datos registrados exitosamente.

5. ESTRUCTURA DEL PROYECTO

La aplicación fue organizada mediante paquetes y capas para facilitar la administración del código.

Estructura principal



Descripción de componentes

Archivo	Función
ConexionBD.java	Conexión con MySQL
VehiculoDAO.java	Inserción de datos
VehiculoServlet.java	Procesamiento HTTP
registrar.jsp	Formulario de registro
resultado.jsp	Confirmación visual
web.xml	Configuración del servlet

6. BASE DE DATOS

Nombre de la Base de Datos

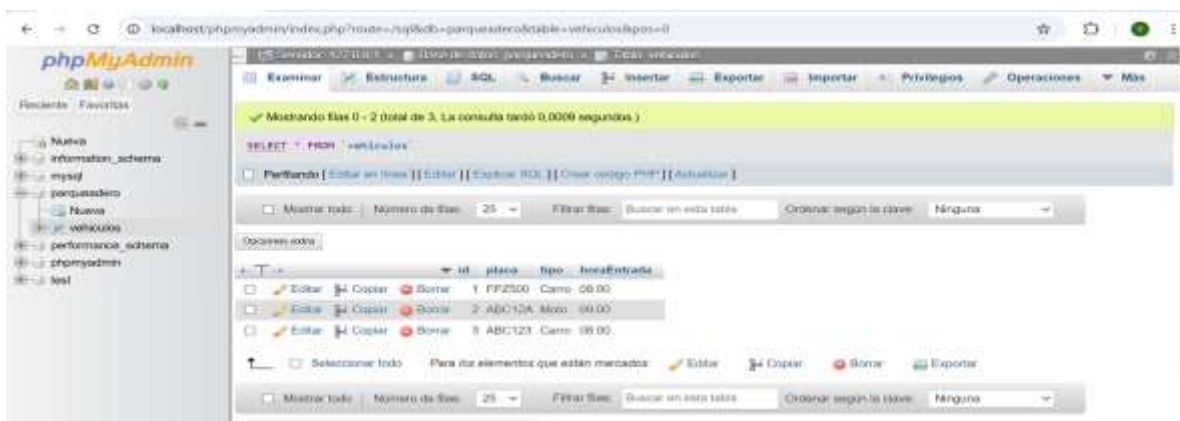
parqueadero

Script SQL utilizado

```
CREATE TABLE vehiculos (
id INT PRIMARY KEY AUTO_INCREMENT,
placa VARCHAR(20),
tipo VARCHAR(20),
horaEntrada VARCHAR(20)
);
```

Descripción de la tabla

Campo	Tipo	Descripción
id	INT	Identificador
placa	VARCHAR(20)	Placa vehículo
tipo	VARCHAR(20)	Tipo vehículo
horaEntrada	VARCHAR(20)	Hora ingreso



7. DESARROLLO DEL SISTEMA

7.1 registrar.jsp

Este archivo contiene el formulario principal utilizado para capturar la información del vehículo.

Funciones principales

- Captura de placa.
- Captura de tipo de vehículo.
- Captura de hora de entrada.
- Envío mediante método POST.



The screenshot shows a web browser window with the address bar displaying "localhost:8080/Parqueaderos/VehiculoServlet". The main content area displays a confirmation message: "Vehiculo Registrado Correctamente". Below this message, the captured data is shown: "Placa:" followed by "ABC123", "Tipo:" followed by "Carro", and "Hora Entrada:" followed by "08:00". At the bottom, there is a blue underlined link that says "Registrar Otro Vehiculo".

← → ↻ ⓘ localhost:8080/Parqueaderos/VehiculoServlet

Vehiculo Registrado Correctamente

Placa:
ABC123

Tipo:
Carro

Hora Entrada:
08:00

[Registrar Otro Vehiculo](#)

7.2 VehiculoServlet.java

El Servlet es el encargado de recibir la información enviada desde el formulario JSP.

Funciones principales

- Recepción de parámetros HTTP.
- Procesamiento de datos.
- Comunicación con DAO.
- Envío de atributos a resultado.jsp.



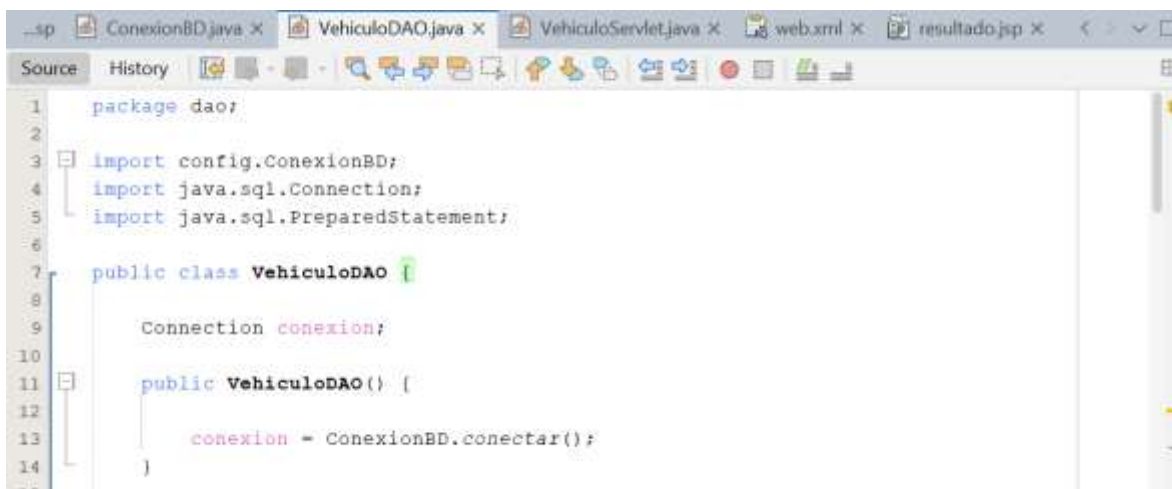
```
1 package servlet;
2
3 import dao.VehiculoDAO;
4
5 import java.io.IOException;
6
7 import jakarta.servlet.ServletException;
8 import jakarta.servlet.http.HttpServlet;
9 import jakarta.servlet.http.HttpServletRequest;
10 import jakarta.servlet.http.HttpServletResponse;
11
12 public class VehiculoServlet extends HttpServlet {
13
14     VehiculoDAO dao = new VehiculoDAO();
15 }
```

7.3 VehiculoDAO.java

La clase DAO permite encapsular la lógica de acceso a datos.

Funciones principales

- Conexión SQL.
- Inserción de registros.
- Uso de PreparedStatement.



```
1 package dao;
2
3 import config.ConexionBD;
4 import java.sql.Connection;
5 import java.sql.PreparedStatement;
6
7 public class VehiculoDAO {
8
9     Connection conexion;
10
11     public VehiculoDAO() {
12
13         conexion = ConexionBD.conectar();
14     }
15 }
```

7.4 ConexionBD.java

Esta clase administra la conexión entre Java y MySQL.

Funciones principales

- Conexión JDBC.
- Driver MySQL.
- Validación de conexión.



The screenshot shows an IDE window with several tabs: ConexionBD.java, VehiculoDAO.java, VehiculoServlet.java, web.xml, and resultado.jsp. The 'Source' tab is active, displaying the code for ConexionBD.java. The code is as follows:

```
1 package config;
2
3 import java.sql.Connection;
4 import java.sql.DriverManager;
5
6 public class ConexionBD {
7
8     public static Connection conectar() {
9
10         Connection con = null;
11
12         try {
13
14             Class.forName("com.mysql.cj.jdbc.Driver");
```

7.5 resultado.jsp

Página encargada de mostrar los datos registrados correctamente.

Funciones principales

- Visualización de datos.
- Confirmación de registro.
- Navegación para nuevo registro.



8. EJECUCIÓN DEL SISTEMA

URL funcional del sistema

<http://localhost:8080/Parqueaderos/registrar.jsp>

Flujo del sistema

Formulario JSP

↓

VehiculoServlet

↓

VehiculoDAO

↓

ConexionBD

↓

MySQL

↓

resultado.jsp

Evidencias

Evidencia 1



← → ↻ ⓘ localhost:8080/Parqueaderos/registrar.jsp

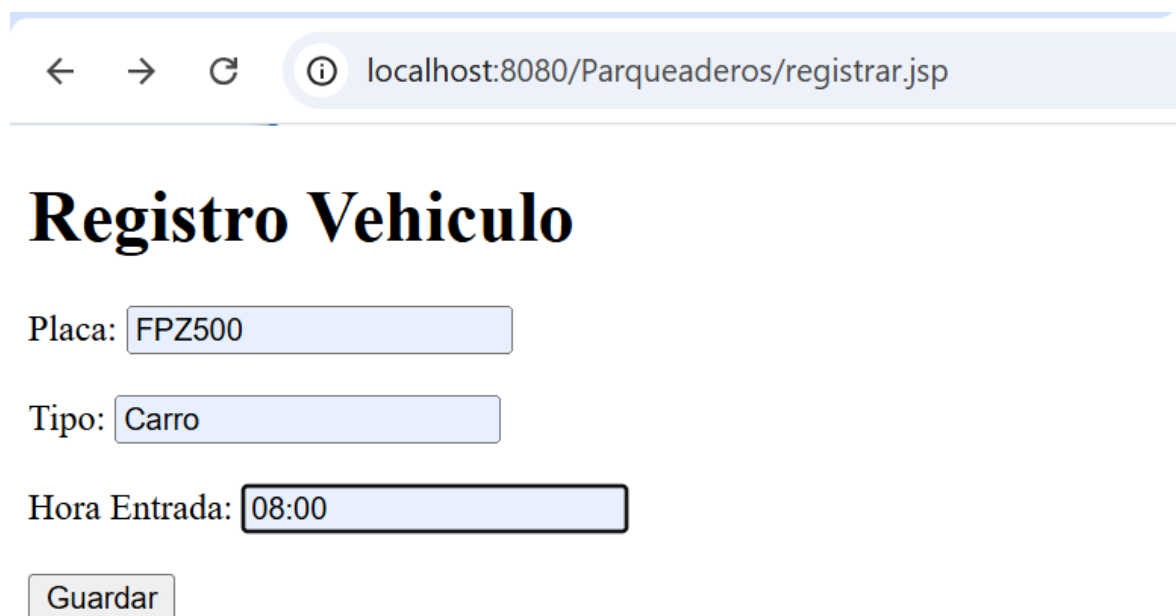
Registro Vehiculo

Placa:

Tipo:

Hora Entrada:

Evidencia 2



A screenshot of a web browser window. The address bar shows 'localhost:8080/Parqueaderos/regarstrar.jsp'. The page title is 'Registro Vehiculo'. Below the title, there are three input fields: 'Placa:' with the value 'FPZ500', 'Tipo:' with the value 'Carro', and 'Hora Entrada:' with the value '08:00'. At the bottom, there is a 'Guardar' button.

← → ↻ ⓘ localhost:8080/Parqueaderos/regarstrar.jsp

Registro Vehiculo

Placa:

Tipo:

Hora Entrada:

Evidencia 3



A screenshot of a web browser window. The address bar shows 'localhost:8080/Parqueaderos/VehiculoServlet'. The page title is 'Vehiculo Registrado Correctamente'. Below the title, there are three labels: 'Placa:', 'Tipo:', and 'Hora Entrada:'. Each label is followed by its corresponding value: 'ABC123', 'Carro', and '08:00'. At the bottom, there is a link 'Registrar Otro Vehiculo'.

← → ↻ ⓘ localhost:8080/Parqueaderos/VehiculoServlet

Vehiculo Registrado Correctamente

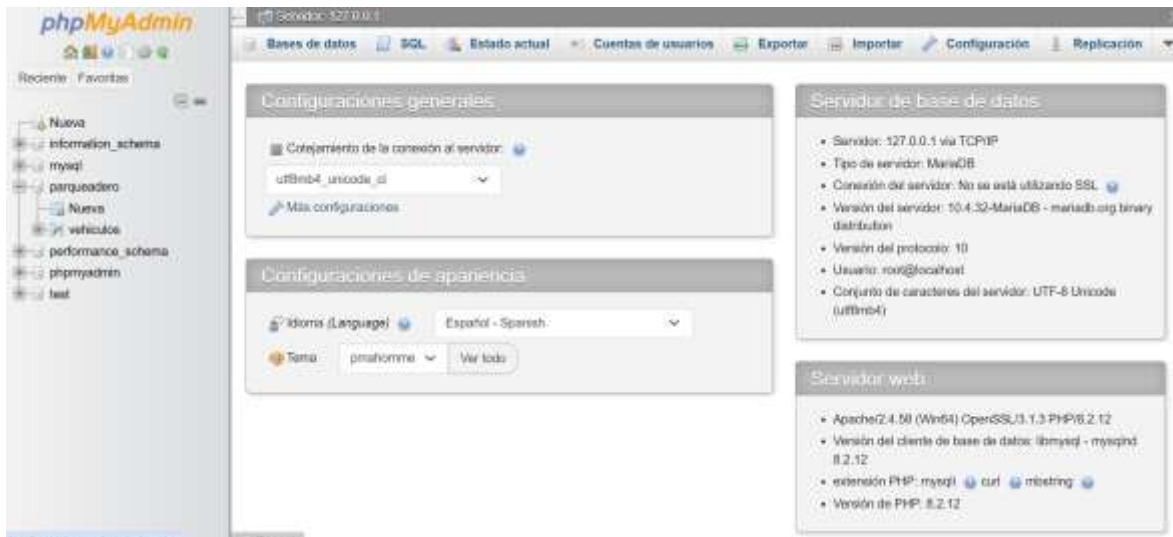
Placa:
ABC123

Tipo:
Carro

Hora Entrada:
08:00

[Registrar Otro Vehiculo](#)

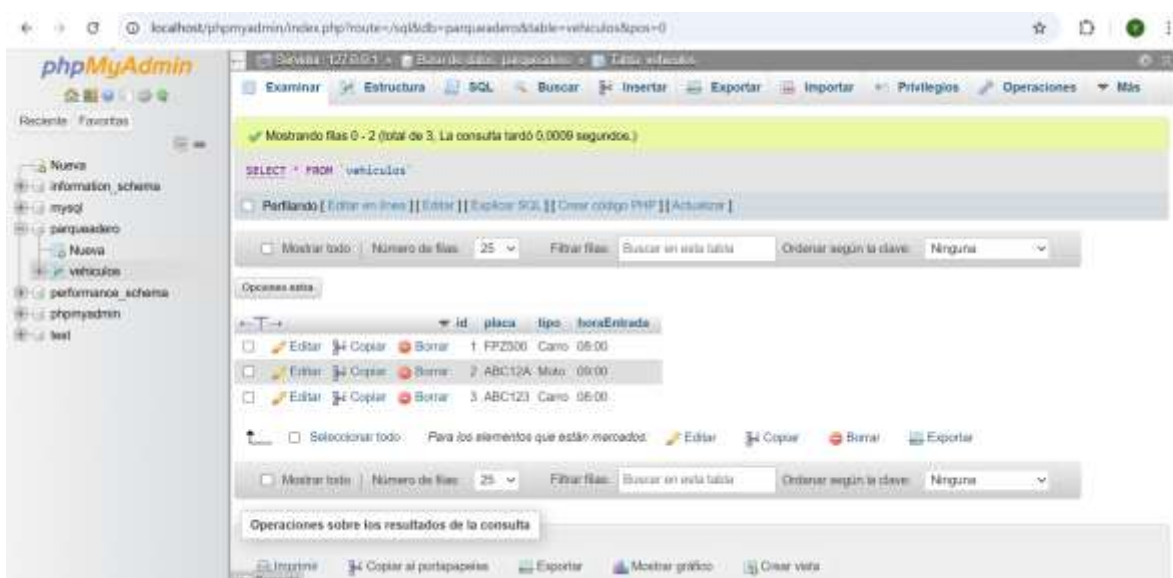
Evidencia 4



Evidencia 5



Evidencia 6



9. DIFICULTADES PRESENTADAS

Durante el desarrollo se presentaron inconvenientes relacionados con el despliegue de la aplicación en Apache Tomcat debido a diferencias en el contexto de publicación del proyecto.

También se presentaron dificultades relacionadas con la configuración del entorno Jakarta EE y la correcta identificación de la ruta de despliegue utilizada por Tomcat.

Posteriormente se solucionaron los inconvenientes mediante la validación del contexto real de despliegue, configuración correcta del web.xml y ajuste de la URL funcional del sistema.

10. SOLUCIÓN IMPLEMENTADA

Se implementó una arquitectura básica MVC utilizando:

- JSP para interfaz.
- Servlets para lógica.
- DAO para acceso a datos.
- JDBC para conexión MySQL.
- Apache Tomcat como servidor web.

Finalmente se validó el correcto almacenamiento de datos mediante pruebas reales en MySQL.

11. RESULTADOS OBTENIDOS

Se logró implementar correctamente un sistema web funcional que permite:

- Registrar vehículos.
- Procesar información HTTP.
- Conectarse a MySQL.
- Almacenar datos.
- Mostrar resultados exitosos.
- Ejecutar la aplicación desde Apache Tomcat.

12. CONCLUSIONES

El desarrollo de esta evidencia permitió aplicar conocimientos relacionados con programación web en Java utilizando tecnologías JSP, Servlets y MySQL.

Además, se fortalecieron habilidades relacionadas con configuración de servidores, despliegue de aplicaciones, conexión JDBC y manejo de bases de datos.

Finalmente se logró construir una aplicación web completamente funcional siguiendo una estructura organizada y aplicando buenas prácticas de desarrollo.